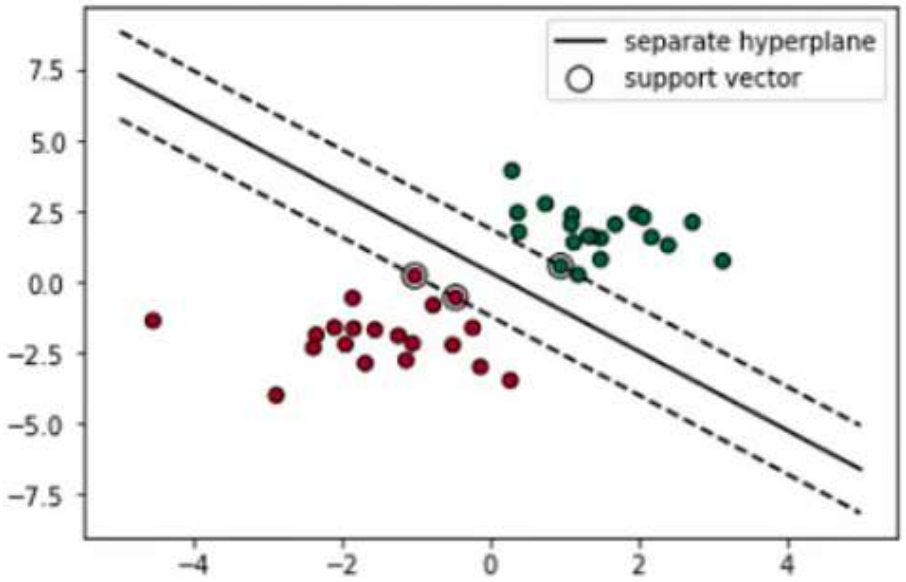
Support Vector Machine (SVM)

Support Vector Machine



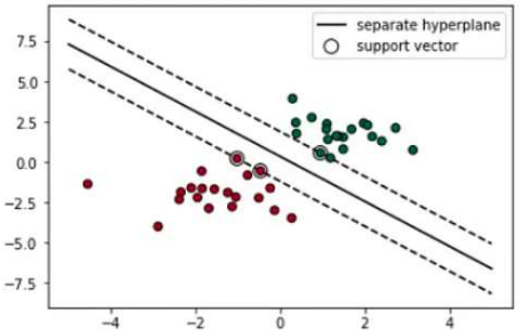
- This figure illustrates the basic idea of a **Support Vector Machine (SVM)** for **binary classification**

The data points

There are two classes:

- **Red points** → Class 1
- **Green points** → Class 2
- The goal of SVM is to find a boundary that **separates the two classes**.
- The **solid black line** is the **separating hyperplane**. For this 2-dimensional example, the hyperplane is simply a **straight line**.

Support Vector Machine

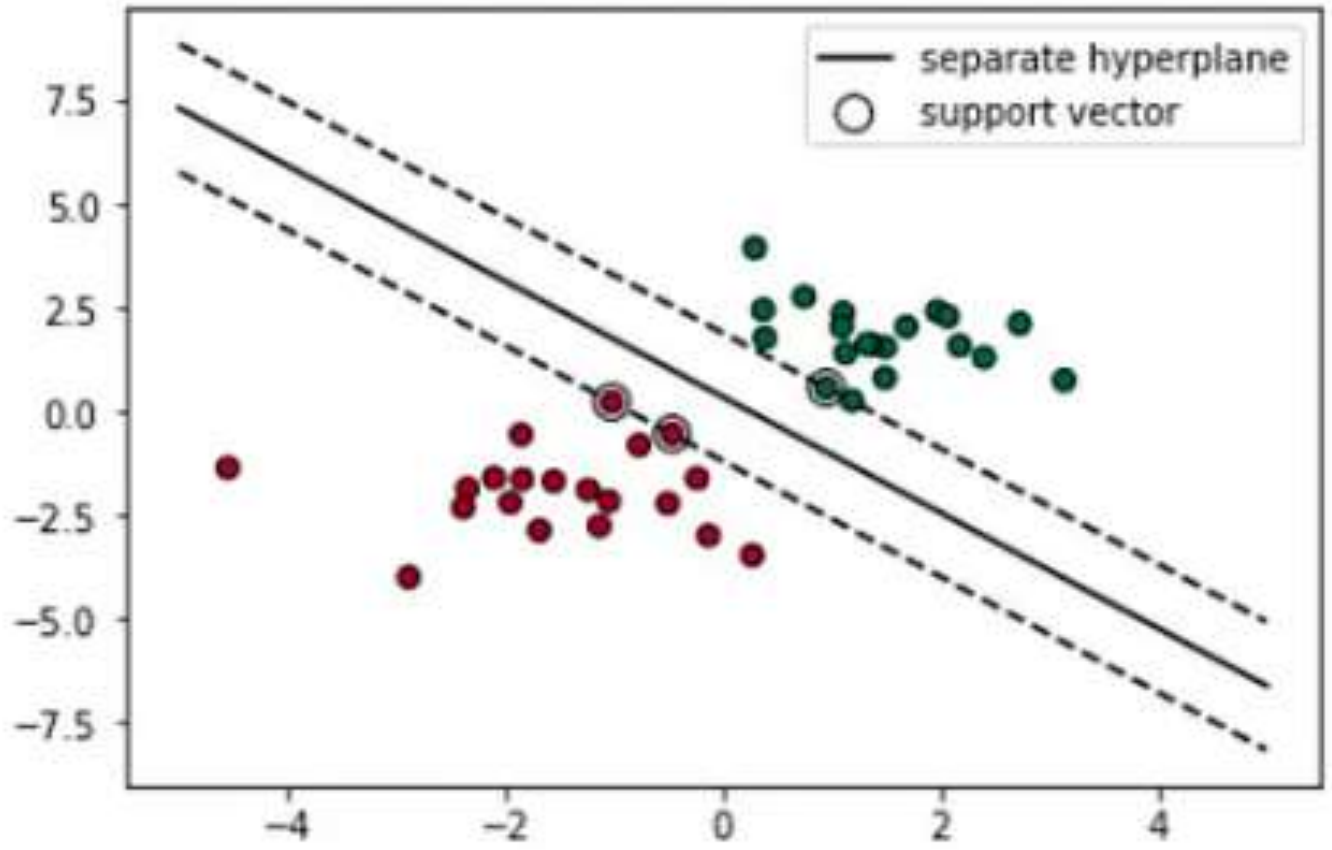


- Cars, building, pedestrians all are dots now.
- Red dots stand for objects on the left side of the street;
- Green dots stand for those on the right.
- The street becomes dashed and solid lines.

- Support Vector Machine (the “Road machine”) is responsible for finding the decision boundary to separate different classes and maximize the margin.

- Margins are the (perpendicular) distances between the line and those dots closest to the line.

Support Vector Machine (Classifier)



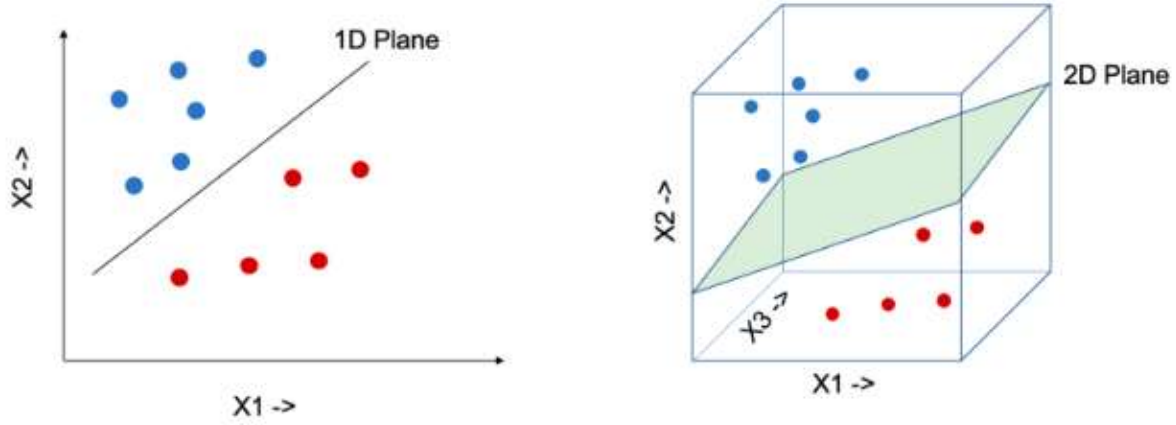
SVM in linear separable cases

- Infinite lines exist to separate the red and green dots
- SVM needs to find the optimal line with the constraint of correctly classifying either class:

1. Follow the constraint: only look into the separate hyperplanes(e.g. separate lines), hyperplanes that classify classes correctly
2. Conduct optimization: pick up the one that maximizes the margin

Hyperplane

- For 2-dimensional space, hyperplane is a line. For a 3-dimensional space, its hyperplane is a plane
- We can use a line to separate data which is in two dimension (Have 2 features x1 and x2).
- Similarly need a 2D plane to separate data in 3 dimension.
- In order to generalize the concepts, we call them hyperplane, instead of line, plane or cube for n dimension of data, where n > 0



Hyperplane

- A hyperplane is a decision boundary that differentiates the two classes in SVM.
- Ex: an email decision boundary might classify an email with over 10 exclamation marks as "spam" and an email with under 10 marks as "not spam.
- A data point falling on either side of the hyperplane can be attributed to different classes.
- The dimension of the hyperplane depends on the number of input features in the dataset.
- If we have 2 input features the hyper-plane will be a line.
- likewise, if the number of features is 3, it will become a two-dimensional plane.

Hyperplane

- In a p-dimensional space, a hyperplane is a flat subspace of hyperplane dimension p − 1.
  - Ex: in two dimensions, a **hyperplane is a flat one-dimensional subspace**—in other words, a line.
  - In three dimensions, **a hyperplane is a flat two-dimensional subspace**—that is, a plane.
  - In p > 3 dimensions, it can be hard to visualize a hyperplane, but the notion of a (p − 1)-dimensional flat subspace still applies

Hyperplane

- In two dimensions, a hyperplane is defined by the equation

$$\beta 0 + \beta 1 X1 + \beta 2 X2 = 0$$

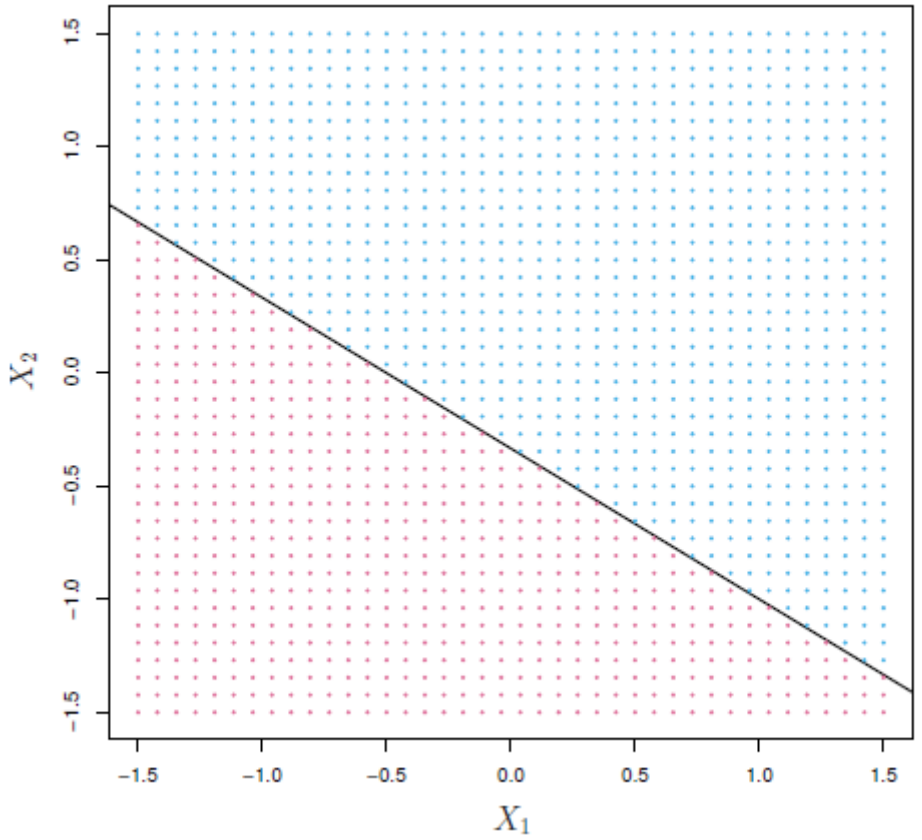
- any $X = (X1, X2)$ for which above equation holds is a point on the hyperplane.
- Equation can be easily extended to the p-dimensional setting:

$$\beta 0 + \beta 1 X1 + \beta 2 X2 + \cdots + \beta p Xp = 0$$

- defines a p-dimensional hyperplane, in the sense that if a point

$X = (X1, X2, \ldots, Xp)$ in p-dimensional space (i.e. a vector of length p) satisfies above equation then X lies on the hyperplane

A 2-dimensional hyperplane: 1 + 2X1 + 3X2 = 0



The hyperplane 1 + 2X1 + 3X2 = 0.
The blue region is the set of points for which 1+2X1 +3X2 >= 0
and the purple region is the set of points for which 1 + 2X1 + 3X2 < 0.

A 2-dimensional hyperplane

- Now, suppose that X does not satisfy $\beta 0 + \beta 1X1 + \beta 2X2 + \cdots + \beta pXp = 0$
- If $\beta 0 + \beta 1X1 + \beta 2X2 + \cdots + \beta pXp > 0$. Then this tells us that X lies to one side of the hyperplane.
- On the other hand, if $\beta 0 + \beta 1X1 + \beta 2X2 + \cdots + \beta pXp < 0$, then X lies on the other side of the hyperplane.
- So we can think of the hyperplane as dividing p-dimensional space into two halves.
- One can easily determine on which side of the hyperplane a point lies by simply calculating the sign of the left hand side of the above equation.

Maximal-Margin Classifier

- The Maximal-Margin Classifier is a hypothetical classifier that best explains how SVM works in practice.
- The numeric input variables (x) in our data (the columns) form an n-dimensional space.
- Ex: for two input variables, this would form a two-dimensional space.
- A hyperplane is a line that splits the input variable space.
- In SVM, a hyperplane is selected to best separate the points in the input variable space by their class, either class 0 or class 1.
- In two-dimensions visualize this as a line and assume that all of our input points can be completely separated by this line. Ex:

$$B0 + (B1 * X1) + (B2 * X2) = 0$$

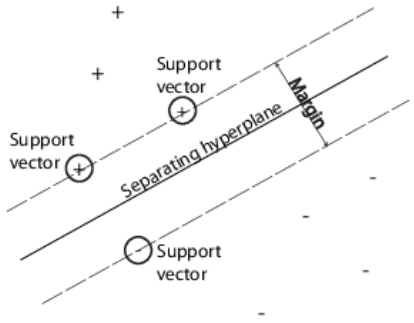
Maximal-Margin Classifier

- Coefficients and B0 are found by the learning algorithm
  - the coefficients (B1 and B2) - determine the slope of the line and
  - B0 – intercept
- X1 and X2 are the two input variables.
- We can make classifications using this line.
- By plugging in input values into the line equation, we can calculate whether a new point is above or below the line.

Maximal-Margin Classifier

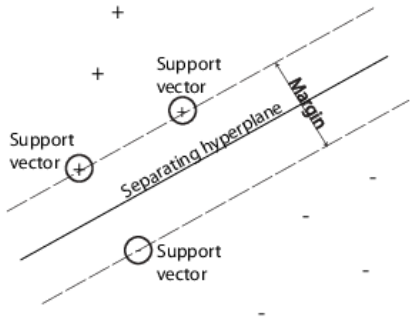
- Above the line, the equation returns a value greater than 0 and the point belongs to the first class (class 0).
- Below the line, the equation returns a value less than 0 and the point belongs to the second class (class 1).
- A value close to the line returns a value close to zero and the point may be difficult to classify.
- If the magnitude of the value is large, the model may have more confidence in the prediction.

Maximal-Margin Hyperplane



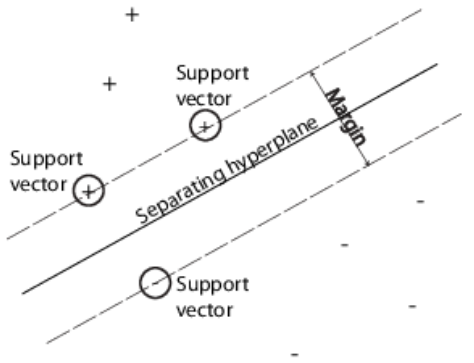
- The distance between the line and the closest data points is referred to as the margin.
- The best or optimal line that can separate the two classes is the line having the largest margin.
- The idea is to maximize the margin, which is the distance between the hyperplane and the closest data points of each category, thus making it easy to distinguish data classes (maximum distances between the two classes)
- This is called the Maximal-Margin hyperplane.

Maximal-Margin Hyperplane



- The margin is calculated as the perpendicular distance from the line to only the closest points.
- Only these points are relevant in defining the line and in the construction of the classifier.
- These points are called the support vectors.
- They support or define the hyperplane.
- The hyperplane is learned from training data using an optimization procedure that maximizes the margin.

Maximal-Margin Classifier



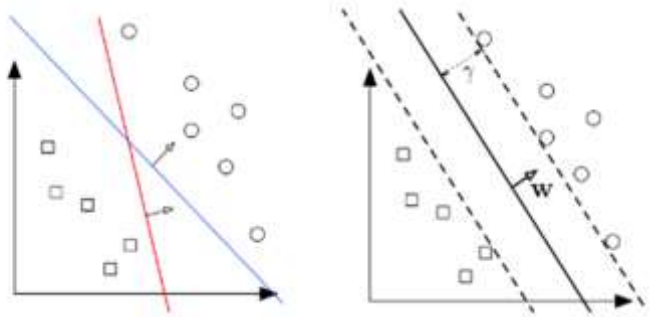
- The support vector machine is a generalization of a simple and intuitive classifier called the **maximal margin classifier**
- Though Maximal-Margin Classifier is simple, this classifier unfortunately cannot be applied to most data sets
- since it requires that the classes be separable by a linear boundary
- The best hyperplane for an SVM means the one with the largest margin between the two classes. Margin means the maximal width of the slab parallel to the hyperplane that has no interior data points

The Output of the SVM

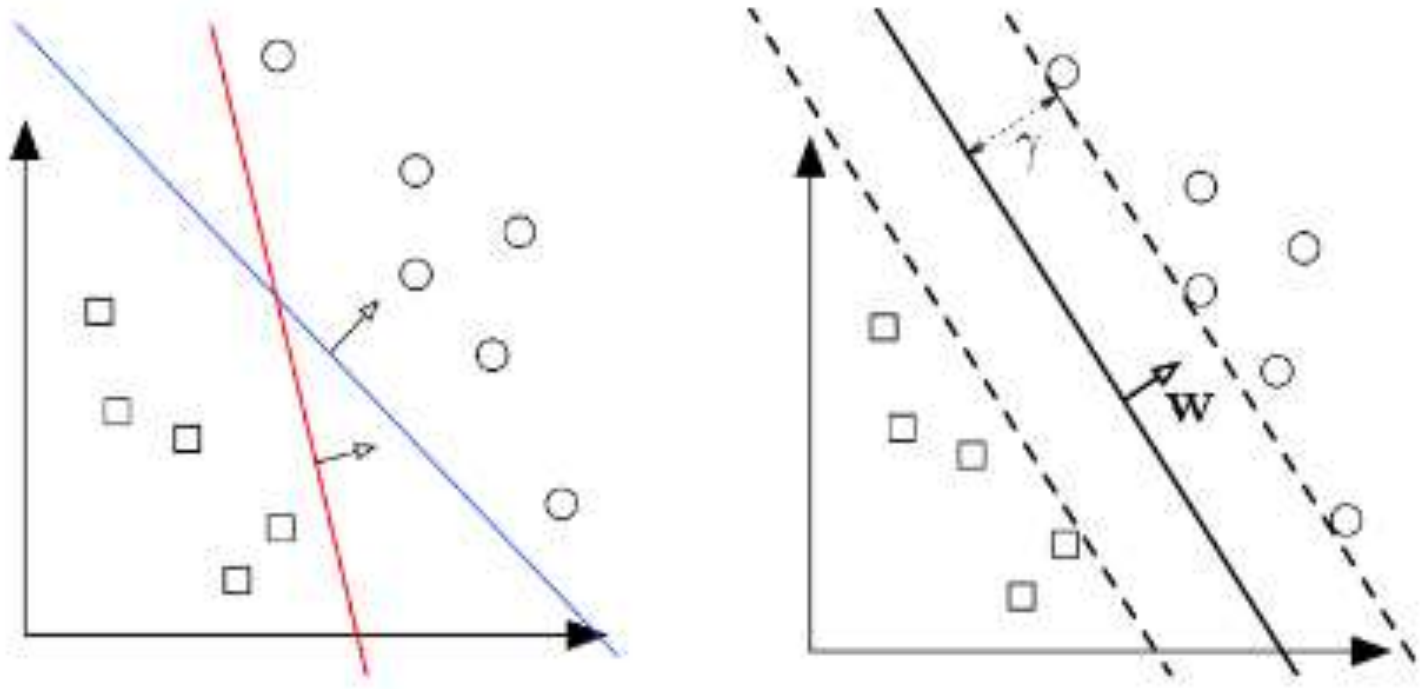
- Once trained, SVMs can classify new, unseen data points by determining which side of the decision boundary they fall on.
- The output of the SVM is the class label associated with the side of the decision boundary

Maximal-Margin hyperplane

- Question: What is the best separating hyperplane for the plot below?
- Typically, if a data set is linearly separable, there are infinitely many separating hyperplanes.
- The one that maximizes the distance to the closest data points from both classes is the best separating hyperplane
- We say it is the hyperplane with maximum margin.



Identify the Maximal-Margin hyperplane

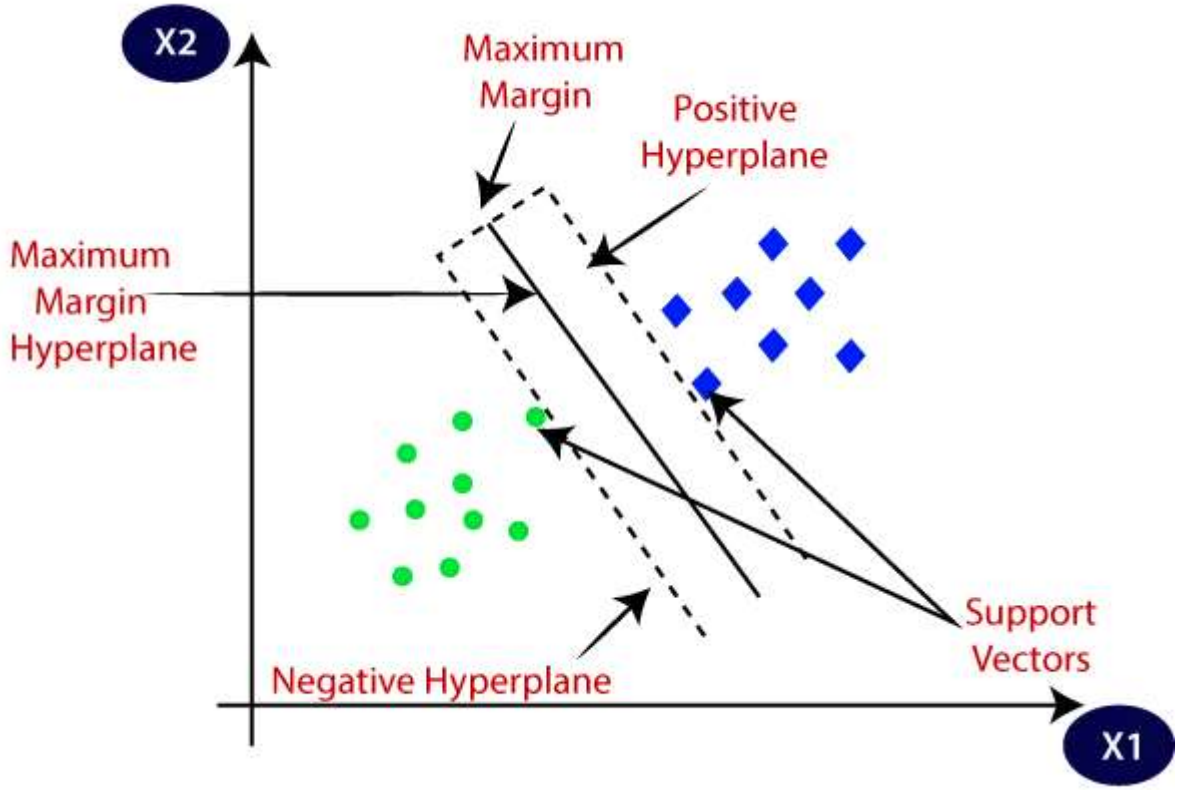


Two different separating hyperplanes for the same data set

(Right:) The maximum margin hyperplane.

The margin, γ is the distance from the hyperplane (solid line) to the closest points in either class (which touch the parallel dotted lines)

Linear SVM



- Linear SVM: Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data
- Such classifier is called as Linear SVM classifier

SVM

- Algorithms that do not make particular assumptions about the kind of mapping function are known as non-parametric algorithms.
- They have the freedom to choose any functional form from the training data.
- So non-parametric models to estimate the mapping function require much more data than parametric ones.
- Non-parametric does not mean that there are no parameters. It means that the parameters are (not only) adjustable but can also change.

SVM

Key distinction between parametric and non-parametric algorithms

- Parametric algorithms have a fixed number of parameters, regardless of the amount of training data.
- For non-parametric, the number of parameters is dependent on the amount of training data.
- The more training data, the greater the number of parameters. A consequence of this is that non-parametric algorithms may take much longer to train.
- K-nearest neighbors is an example of a non-parametric algorithm. Its predictions are based on the k most similar training patterns for a new instance of data.

SVM

- Support Vector Machines are a flexible *nonparametric machine learning algorithm* (linear SVM is parametric based)
- SVMs that are not constrained by a set number of parameters are considered non-parametric

SVM – Training data 1

- Support Vector Machine algorithm on Training data - steps
- Use sub-gradient descent to update the coefficients for an SVM model.
- iterate the sub-gradient descent algorithm to learn an SVM model for training data.
- make predictions given a learned SVM model.

SVM – Training data 1

x1= [2.327868056,3.032830419, 4.485465382, 3.684815246, 2.283558563, 7.807521179, 6.132998136, 7.514829366, 5.502385039, 7.432932365]

x2= [2.458016525, 3.170770366,3.696728111, 3.846846973, 1.853215997, 3.290132136, 2.140563087, 2.107056961, 1.404002608, 4.236232628]

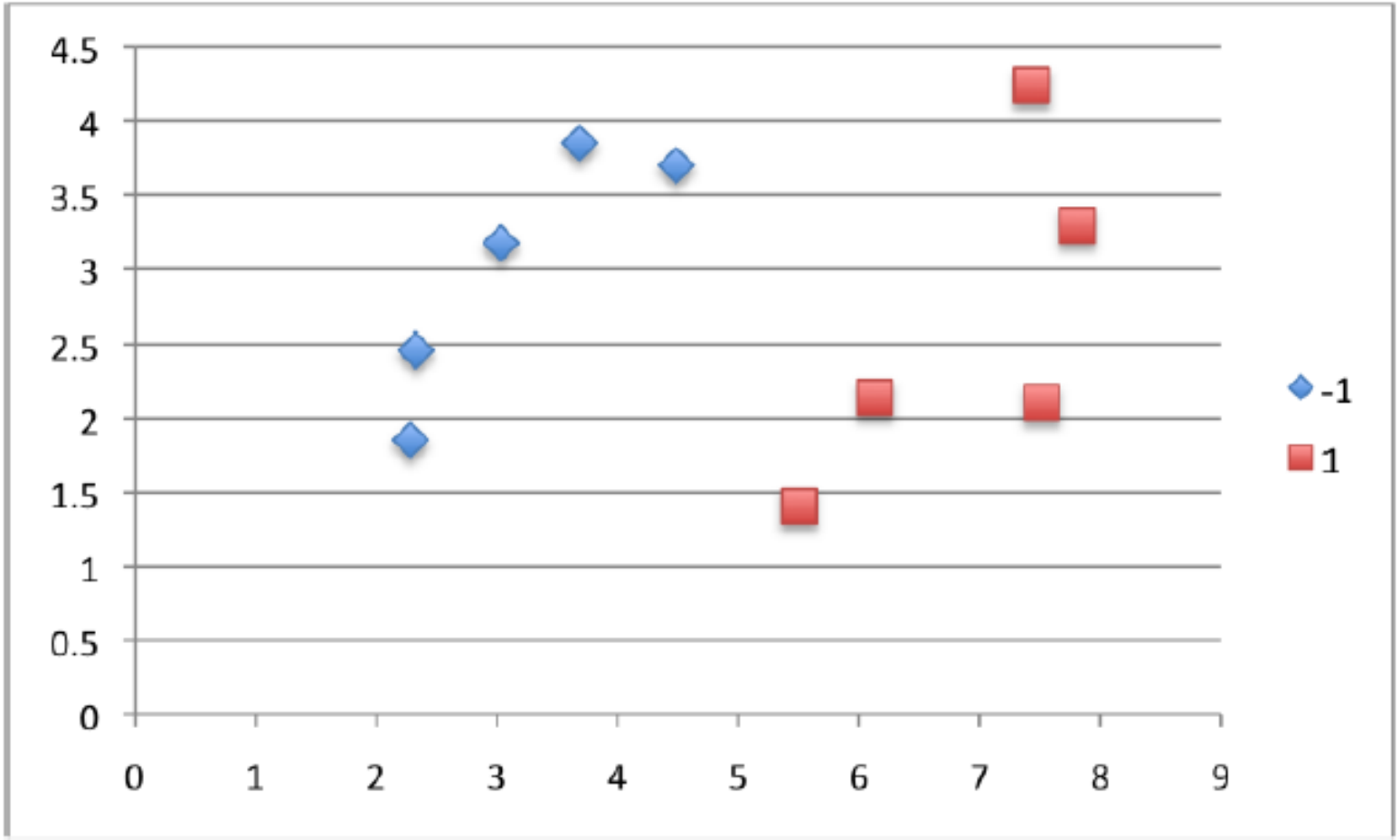
y=[-1, -1, -1, -1, -1, 1, 1, 1, 1, 1]

SVM – Training data 1

Are classes linearly separable?

- This means that a straight line can be drawn to separate the classes.
- In that case, we can implement an SVM with a linear kernel (straight line).
- Plot the scatter graph

visualization of a scatter plot of the dataset



Training SVM With Gradient Descent

- Form of Linear SVM Model
- The Linear SVM model is a line
- The goal of the learning algorithm is to find values for the coefficients that best separates the classes.
- B0 + (B1 * X1) + (B2 * X2) = 0
  - B0, B1 and B2 are the coefficients and
  - X1 and X2 are the input variables
- This will be the form of the equation but with one modification
- We drop the bias term (B0) also called the offset or the intercept.
- Ex: B0 = 0
- (B1 * X1) + (B2 * X2) = 0
- This means that the line will pass through the origin (X1 = 0 and X2 = 0)
- Note: For the sake of simplicity, the bias term was dropped but it could be included.

SVM Optimization Method

- The optimization algorithm to find the coefficients can be stated as a quadratic programming problem (not used)
- Approach that can be used to discover the coefficient values for Linear SVM is sub-gradient descent.
- In this method a random training pattern is selected for each iteration and used to update the coefficients.
- After a large number of iterations the algorithm will settle on a stable set of coefficients.
- The coefficient update equation
- First an output value is calculated
- output = Y * (B1 * X1) + (B2 * X2)

SVM Optimization Method

- Two different update procedures are used depending on the output value.
- If the output value is greater than 1
- it suggests that the training pattern was not a support vector.
- This means that the instance was not directly involved in calculating the output, in which case the weights are slightly decreased:
- $b = (1 – 1/t) * b$
  - b is the weight that is being updated (such as B1 or B2),
  - t is the current iteration (e.g. 1 for the first update, 2 for the second and so on).

SVM Optimization Method

- If the output is less than 1
- it is assumed that the training instance is a support vector and must be updated to better explain the data.

$$b = (1 - \frac{1}{t}) \times b + \frac{1}{lambda \times t} \times (y \times x)$$

  - b is the weight that is being updated,
  - t is the current iteration and
  - lambda is a parameter to the learning algorithm.
  - The lambda is a learning parameter and often set to a very small values such as 0.0001 or smaller.

SVM Optimization Method

- The procedure is repeated until the error rate drops to a desirable level or for a very large fixed number of iterations.
- Smaller learning rates require much longer training times.
- The number of iterations is a downside to this learning algorithm.

SVM – Training data 1

- Lambda Value:
- Note: We will use a very large lambda value: lambda = 0.4.
- This is unusually large and will force a lot of change on each update.
- Normally lambda values are very small.

Learning Iteration #1

- set the coefficients to 0.0.
- B1 = 0.0
- B2 = 0.0
- t = 1: keep track of which iteration we are on
- Train the model using the order of the training patterns.
- Note: Ideally, the order of the patterns would be randomized to avoid the learning algorithm getting stuck.

SVM – Training data 1

- Instance: X1 = 2.327868056, X2 = 2.458016525, Y = -1.
- calculate the output value for this iteration
- output = Y * (B1 * X1) + (B2 * X2)
- output = -1 * (0.0 * 2.327868056) + (0.0 * 2.458016525)
- output = 0.0
- The output is less than 1.0, therefore we use the complex update procedure that assumes the training pattern is a support vector

Iteration #1

x1= 2.327868056

x2= 2.458016525

y= -1

b1 = -4.655736112

b2= -4.91603305

output= 0.0

t= 1

SVM – Training data 1

x1= [2.327868056,3.032830419, 4.485465382, 3.684815246, 2.283558563, 7.807521179, 6.132998136, 7.514829366, 5.502385039, 7.432932365]

x2= [2.458016525, 3.170770366,3.696728111, 3.846846973, 1.853215997, 3.290132136, 2.140563087, 2.107056961, 1.404002608, 4.236232628]

y=[-1, -1, -1, -1, -1, 1, 1, 1, 1, 1]

SVM Program

```
b1=0
b2=0
t=0
rate = 0.5
for epoch in range (0, 20):
    for i in range(0, 10):
        t += 1
        output= y[i]*b1*x1[i] + b2 * x2[i]
        if output < 1:
            b1 = (1-1/t)*b1+(1/(rate*t))*(y[i]*x1[i])
            b2 = (1-1/t)*b2+(1/(rate*t))*(y[i]*x2[i])
        else:
            b1 = (1-1/t)*b1
            b2 = (1-1/t)*b2
        print(i+1, " ", "x1=", x1[i], " ", "x2=", x2[i], " ", "y=", y[i], " " , "b1 = ", b1, " ", "b2= ", b2, " ","output=", output, "t=",
        t)
```

Iteration #1 - #10- Epoch 1

1 x1= 2.327868056 x2= 2.458016525 y= -1 b1 = -4.655736112 b2= -4.91603305 output= 0.0 t= 1

2 x1= 3.032830419 x2= 3.170770366 y= -1 b1 = -5.3606984749999995 b2= -5.628786891000001 output= -1.4675538099062084 t= 2

3 x1= 4.485465382 x2= 3.696728111 y= -1 b1 = -3.5737989833333335 b2= -3.752524594000001 output= 3.2371327021646934 t= 3

4 x1= 3.684815246 x2= 3.846846973 y= -1 b1 = -4.5227568605 b2= -4.737816932000001 output= -1.26659889561099 t= 4

5 x1= 2.283558563 x2= 1.853215997 y= -1 b1 = -3.6182054884000006 b2= -3.790253545600001 output= 1.5477820279219099 t= 5

6 x1= 7.807521179 x2= 3.290132136 y= 1 b1 = -0.4126641806666673 b2= -2.061833909333334 output= -40.71965097462355 t= 6

7 x1= 6.132998136 x2= 2.140563087 y= 1 b1 = 1.3985730268571421 b2= -1.1556967545714292 output= -6.944354208666477 t= 7

8 x1= 7.514829366 x2= 2.107056961 y= 1 b1 = 1.2237513984999993 b2= -1.0112346602500004 output= 8.074918761196718 t= 8

9 x1= 5.502385039 x2= 1.404002608 y= 1 b1 = 1.0877790208888882 b2= -0.8988752535555559 output= 5.313775286270728 t= 9

10 x1= 7.432932365 x2= 4.236232628 y= 1 b1 = 0.9790011187999994 b2= -0.8089877282000003 output= 4.277543212719209 t= 10

SVM Program

```
yCorrect = 0
for i in range(0, 10):
    output= b1*x1[i] + b2 * x2[i]
    if output < 0:
        yPredicted = -1
    else:
        yPredicted = 1
    if (y[i] == yPredicted):
        yCorrect += 1
    print(i+1, " ", "x1=", x1[i], " ", "x2=", x2[i], " ", "y=", y[i], " " , "b1 = ", b1, " ", "b2= ", b2,
    " ","output=", output, "yPredicted=",yPredicted)
accuracy = (yCorrect/10)*100
print("epoch = ", epoch+1, "accuracy=", accuracy)
```

Accuracy at the end of Epoch 1

1 x1= 2.327868056 x2= 2.458016525 y= -1 b1 = 0.9790011187999994 b2= -0.8089877282000003 output= 0.29048022680497 yPredicted= 1

2 x1= 3.032830419 x2= 3.170770366 y= -1 b1 = 0.9790011187999994 b2= -0.8089877282000003 output= 0.4040300582974474 yPredicted= 1

3 x1= 4.485465382 x2= 3.696728111 y= -1 b1 = 0.9790011187999994 b2= -0.8089877282000003 output= 1.4006679510256976 yPredicted= 1

4 x1= 3.684815246 x2= 3.846846973 y= -1 b1 = 0.9790011187999994 b2= -0.8089877282000003 output= 0.495386254984977 yPredicted= 1

5 x1= 2.283558563 x2= 1.853215997 y= -1 b1 = 0.9790011187999994 b2= -0.8089877282000003 output= 0.7363773887453904 yPredicted= 1

6 x1= 7.807521179 x2= 3.290132136 y= 1 b1 = 0.9790011187999994 b2= -0.8089877282000003 output= 4.981895447115236 yPredicted= 1

7 x1= 6.132998136 x2= 2.140563087 y= 1 b1 = 0.9790011187999994 b2= -0.8089877282000003 output= 4.2725227679214015 yPredicted= 1

8 x1= 7.514829366 x2= 2.107056961 y= 1 b1 = 0.9790011187999994 b2= -0.8089877282000003 output= 5.652443132837703 yPredicted= 1

9 x1= 5.502385039 x2= 1.404002608 y= 1 b1 = 0.9790011187999994 b2= -0.8089877282000003 output= 4.251020229016582 yPredicted= 1

10 x1= 7.432932365 x2= 4.236232628 y= 1 b1 = 0.9790011187999994 b2= -0.8089877282000003 output= 3.8497888914472886 yPredicted= 1

epoch = 1 accuracy= 50.0

SVM - More Iterations

- Repeat this process for the rest of the dataset
- One pass through the dataset is called an epoch.
- Repeat the process for a further 20 epochs for a total of 200 iterations (20 epochs * 10 updates per epoch).
- It is possible to keep track of the loss or the accuracy of the model for each epoch to check whether the algorithm is converging
- Plot the accuracy for the model at the end of each epoch

Make Predictions with SVM Model

- Now that we have coefficients for the line:
- we can make predictions for the training data and
- could be adapted to make predictions for new data.
- Predictions can be made using the following equation:
- output = (B1 * X1) + (B2 * X2)
- Y = -1 IF output < 0
- Y = +1 IF output > 0

Accuracy at the end of Epoch #1 - #20

1 50.0

2 80.0

3 80.0

4 80.0

5 80.0

6 100.0

7 100.0

8 90.0

9 100.0

10 100.0

11 100.0

12 100.0

13 100.0

14 100.0

15 100.0

16 100.0

17 100.0

18 90.0

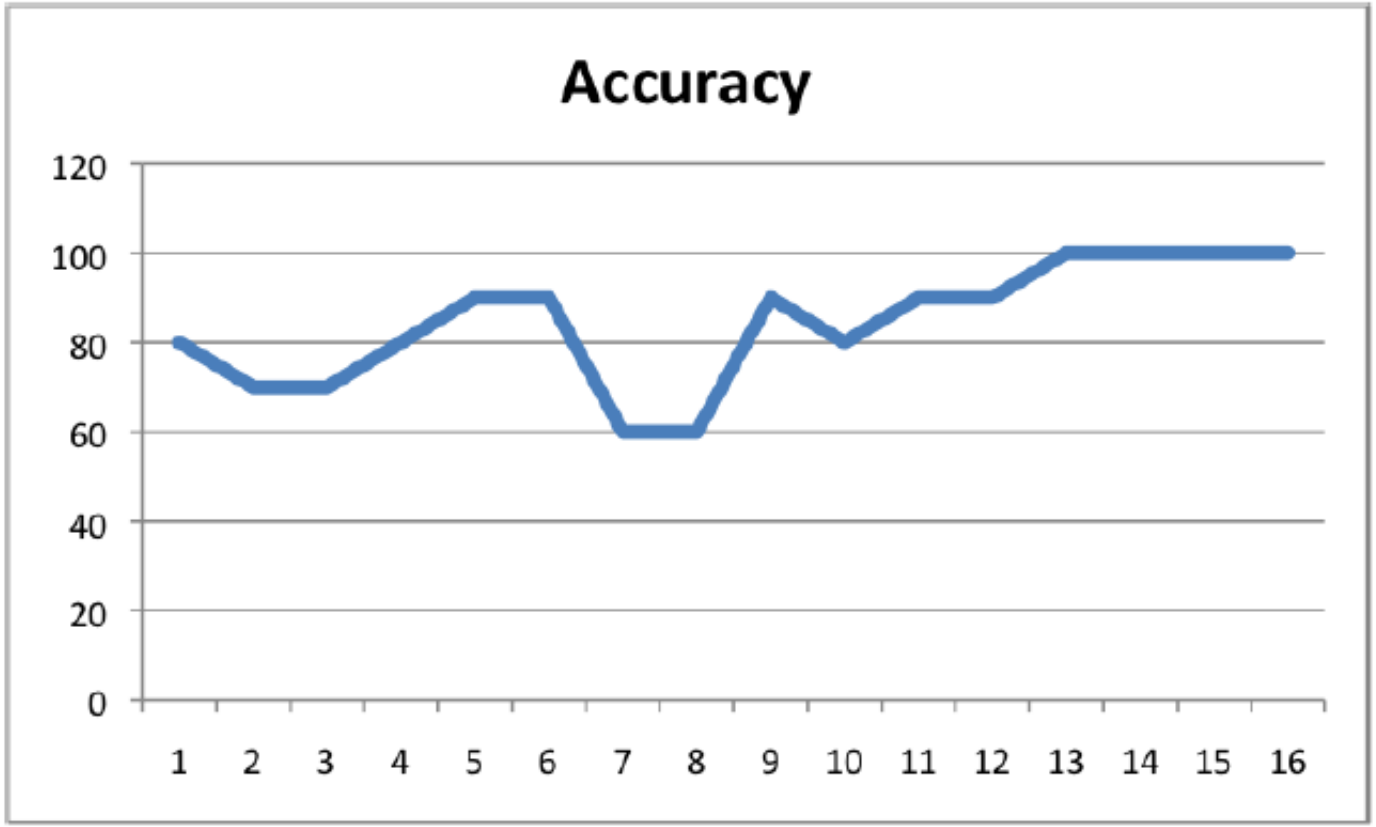
19 100.0

20 100.0

SVM - More Iterations

- We achieve an accuracy of 100% on the training data at the end of 20 epochs
- Final values for the coefficients at the end of 20 epochs:
- b1 = 0.7379828004500003 b2= -1.2083184512899998
- The form of the learned hyperplane is
- 0 + (0.7379828004500003 * X1) + (-1.2083184512899998 * X2) = 0

A Support Vector Machine Model Accuracy – (Not for Training Data 1)



Derivation

- https://machinelearningmastery.com/method-of-lagrange-multipliers-the-theory-behind-support-vector-machines-part-1-the-separable-case/

- https://machinelearningmastery.com/method-of-lagrange-multipliers-the-theory-behind-support-vector-machines-part-2-the-non-separable-case/

- https://machinelearningmastery.com/method-of-lagrange-multipliers-the-theory-behind-support-vector-machines-part-3-implementing-an-svm-from-scratch-in-python/

SVM Example – Classify fruit – Ex 2

- X = [[5, 7], [7, 5], [2, 8], [3, 6], [8, 1], [7, 3], [1, 3], [2, 2], [9, 6], [6, 7]]
- y = [1, 1, 1, 1, -1, -1, -1, -1, 1, 1]

SVM Example – Classify fruit

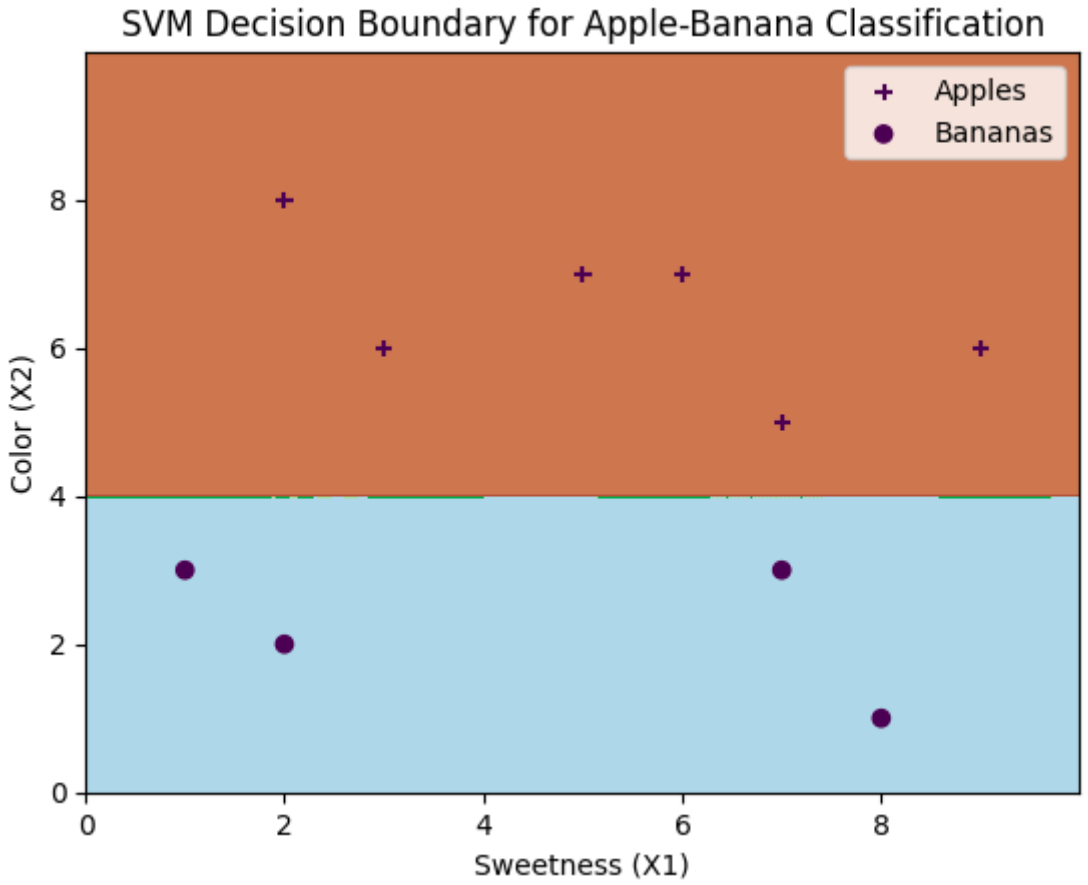
```
from sklearn import svm
 # Create a Support Vector Machine with a linear kernel
clf = svm.SVC(kernel='linear’)
# Define the features and labels from the dataset
X = [[5, 7], [7, 5], [2, 8], [3, 6], [8, 1], [7, 3], [1, 3], [2, 2], [9, 6], [6, 7]]
y = [1, 1, 1, 1, -1, -1, -1, -1, 1, 1]
# Fit the SVM to the data
clf.fit(X, y)
new_fruit = [[4, 5]]
prediction = clf.predict(new_fruit)
print("Prediction:", prediction[0])
 # Output: 1 (Apple)
```

SVM Example – Classify fruit

```
import numpy as np
import matplotlib.pyplot as plt
# Create a mesh to plot the decision boundary
h = 0.01  # Step size in the mesh
x_min, x_max = 0, 10
y_min, y_max = 0, 10
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
np.arange(y_min, y_max, h))
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
```

```
# Plot the decision boundary
plt.contourf(xx, yy, Z, cmap=plt.cm.Paired, alpha=0.8)
# Scatter plot the data points
plt.scatter([5, 7, 2, 3, 9, 6], [7, 5, 8, 6, 6, 7], c=[1, 1, 1, 1, 1, 1], marker='+', label='Apples’)
plt.scatter([8, 7, 1, 2], [1, 3, 3, 2], c=[-1, -1, -1, -1], marker='o', label='Bananas’)
# Labels and legend
plt.xlabel('Sweetness (X1)’)
plt.ylabel('Color (X2)’)
plt.legend(loc='upper right’)
plt.title('SVM Decision Boundary for Apple-Banana Classification’)
plt.show()
```

SVM Example – Classify fruit



Support Vector Machine (SVM)

- A Support Vector Machine (SVM) is a powerful and versatile Machine Learning model
- Capable of performing linear or nonlinear classification, regression, and even outlier detection.
- SVMs are particularly well suited for classification of complex small- or medium-sized datasets

How to Learn a SVM Model

- The SVM model needs to be solved using a numerical optimization procedure to search for the coefficients of the hyperplane.
- This is inefficient and is not the approach used in widely used SVM implementations like LIBSVM.
- A variation of gradient descent called sub-gradient descent is used
- There are specialized optimization procedures that re-formulate the optimization problem to be a Quadratic Programming problem.
- The most popular method for fitting SVM is the Sequential Minimal Optimization (SMO) method that is very efficient.
- It breaks the problem down into sub-problems that can be solved analytically (by calculating) rather than numerically (by searching or optimizing).

SVM Ahead

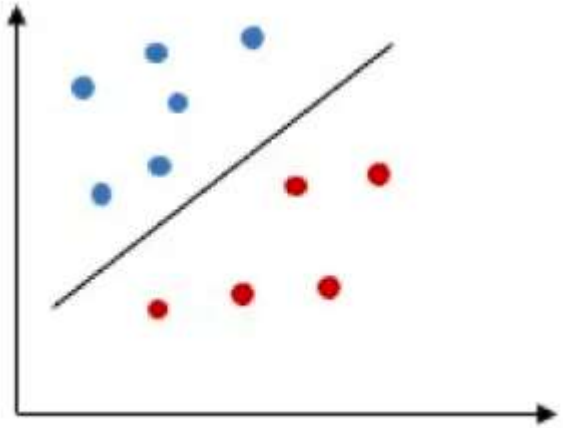
- The Maximal-Margin Classifier provides a simple theoretical model for understanding SVM.
- The Soft Margin Classifier is a modification of the Maximal-Margin Classifier to relax the margin to handle noisy class boundaries in real data.
- Support Vector Machines learning algorithm can be reformulated as a dot-product kernel
- Other kernels like Polynomial and Radial can be used.
- Numerical optimization can be used to learn the hyperplane
- Efficient implementations use an alternate optimization scheme called Sequential Minimal Optimization.

Preparing Data For SVM

- How to best prepare our training data when learning an SVM model ?
- Numerical Inputs: SVM assumes that inputs are numeric.
- For categorical inputs, it is needed to covert to binary dummy variables (one variable for each category).
- Binary Classification: Basic SVM is intended for binary (two-class) classification problems.
- Extensions have been developed for regression and multiclass classification.

Linear SVM – Hard Margin Classifier

- So far, we have see the concept of SVM by classifying perfectly separated dataset (linear classification).
- This is also called “Linear SVM – Hard Margin Classifier”.
- Linear SVM is a generalization of Maximal Margin Classifier.
- Note that Maximal Margin Classifier does not have any practical use and its a theoretical concept



Linear SVM – Soft Margin Classifier

- We can extend our concept of Hard Margin Classifier to solve for dataset where there are some outliers.
- In this case all of the data points cant be separated using a straight line, there will be some miss-classified points.
- This is similar of adding regularization to a regression model.

